РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № *2*

дисциплина: Архитектура компьютера

Студент: Парчиев Султан Багаудинович

Группа: НММ бд-03-24

МОСКВА

2024 г.

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**

4 Выполнение лабораторной работы **8**

5 Выводы **9**

Список литературы **10**

Список иллюстраций

4.1 Рис. 1: Заполнение данных учетной записи GitHub **8**

4.1 Рис. 2: Аккаунт GitHub **8**

4.2 Рис. 3: Предварительная конфигурация git **8**

4.2 Рис. 4: Настройка кодировки **8**

4.2 Рис. 5: Заполнение данных учетной записи GitHub **9**

4.2 Рис. 6: Заполнение данных учетной записи GitHub **9**

4.2 Рис. 7: Заполнение данных учетной записи GitHub **9**

4.2 Рис. 8: Заполнение данных учетной записи GitHub **9**

4.1 Рис. 9: Заполнение данных учетной записи GitHub **10**

4.1 Рис. 10: Заполнение данных учетной записи GitHub **10**

4.1 Рис. 11: Заполнение данных учетной записи GitHub **8**

4.1 Рис. 12: Заполнение данных учетной записи GitHub **8**

4.1 Рис. 13: Заполнение данных учетной записи GitHub **8**

4.1 Рис. 14: Заполнение данных учетной записи GitHub **8**

4.1 Рис. 15: Заполнение данных учетной записи GitHub **8**

4.1 Рис. 16: Заполнение данных учетной записи GitHub **8**

4.1 Рис. 17: Заполнение данных учетной записи GitHub **8**

4.1 Рис. 18: Заполнение данных учетной записи GitHub **8**

4.1 Рис. 19: Заполнение данных учетной записи GitHub **8**

4.1 Рис. 20: Заполнение данных учетной записи GitHub **8**

4.1 Рис. 21: Заполнение данных учетной записи GitHub **8**

4.1 Рис. 22: Заполнение данных учетной записи GitHub **8**

4.1 Рис. 23: Заполнение данных учетной записи GitHub **8**

1 Цель работы

Целью данной работы является изучить идеологию и применение средств контроля версий, а также приобрести практические навыки по работе с системой git.

2 Задание

1. Настройка GitHub.
2. Базовая настройка Git.
3. Создание SSH-ключа.
4. Создание рабочего пространства и репозитория курса на основе шаблона.
5. Создание репозитория курса на основе шаблона.
6. Настройка каталога курса.
7. Выполнение заданий для самостоятельной работы.

3 Теоретическое введение

Системы контроля версий (Version Control System, VCS) применяются при работе нескольких человек над одним проектом. Обычно основное дерево проекта хранится в локальном или удалённом репозитории, к которому настроен доступ для участников проекта. При внесении изменений в содержание проекта система контроля версий позволяет их фиксировать, совмещать изменения, произведённые разными участниками проекта, производить откат к любой более ранней версии проекта, если это требуется. В классических системах контроля версий используется централизованная модель, предполагающая наличие единого репозитория для хранения файлов. Выполнение большинства функций по управлению версиями осуществляется специальным сервером. Участник проекта (пользователь) перед началом работы посредством определённых команд получает нужную ему версию файлов. После внесения изменений пользователь размещает новую версию в хранилище. При этом предыдущие версии не удаляются из центрального хранилища и к ним можно вернуться в любой момент. Сервер может сохранять не полную версию изменённых файлов, а производить так называемую дельта-компрессию — сохранять только изменения между последовательными версиями, что позволяет уменьшить объём хранимых данных. Системы контроля версий поддерживают возможность отслеживания и разрешения конфликтов, которые могут возникнуть при работе нескольких человек над одним файлом. Можно объединить изменения, сделанные разными участниками, вручную выбрать нужную версию, отменить изменения вовсе или заблокировать файлы для изменения. В зависимости от настроек блокировка не позволяет другим пользователям получить рабочую копию или препятствует изменению рабочей копии файла средствами файловой системы ОС, обеспечивая таким образом привилегированный доступ только одному пользователю, работающему с файлом. Системы контроля версий также могут обеспечивать дополнительные, более гибкие функциональные возможности. Например, они могут поддерживать работу с несколькими версиями одного файла, сохраняя общую историю изменений до точки ветвления версий и собственные истории изменений каждой ветви. Обычно доступна информация о том, кто из участников, когда и какие изменения вносил. Обычно такого рода информация хранится в журнале изменений, доступ к которому можно ограничить. В отличие от классических, в распределённых системах контроля версий центральный репозиторий не является обязательным. Среди классических VCS наиболее известны CVS, Subversion, а среди распределённых — Git, Bazaar, Mercurial. Принципы их работы схожи, отличаются они в основном синтаксисом используемых в работе команд. Система контроля версий Git представляет собой набор программ командной строки. Доступ к ним

можно получить из терминала посредством ввода команды git с различными опциями. Благодаря тому, что Git является распределённой системой контроля версий, резервную копию локального хранилища можно сделать простым копированием или архивацией. Работа пользователя со своей веткой начинается с проверки и получения изменений из центрального репозитория (при этом в локальное дерево до начала этой процедуры не должно было вноситься изменений). Затем можно вносить изменения в локальном дереве и/или ветке. После завершения внесения какого-то изменения в файлы и/или каталоги проекта необходимо разместить их в центральном репозитории.

4 Выполнение лабораторной работы

4.1 Настройка GitHub

Создаю учетную запись на сайте GitHub (рис. 1). Далее я заполнил основные данные учетной записи.

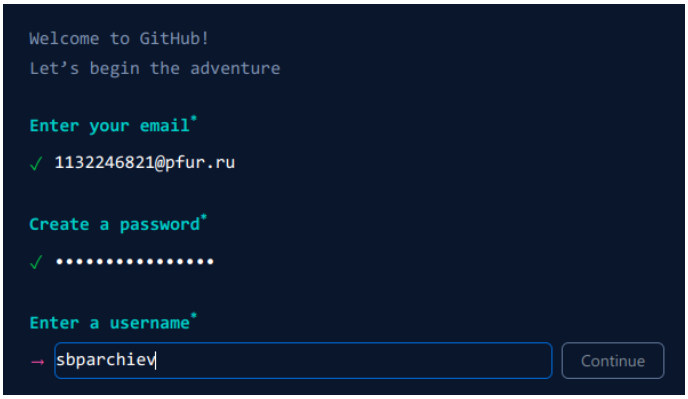

Рис. 1: Заполнение данных учетной записи GitHub

Аккаун создан (рис.2).

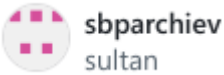

Рис. 2: Аккаунт GitHub

4.2 Базовая настройка Git

Открываю виртуальную машину, затем открываю терминал и делаю предварительную конфигурацию git. Ввожу команду git config –global user.name “”, указывая свое имя и команду git config –global user.email “work@mail”, указывая в ней электронную почту владельца, то есть мою (рис. 3).

```
sultan@sultan-BOHB-WAX9:~$ git config --global user.name '<sbparchiev>'
sultan@sultan-BOHB-WAX9:~$ git config --global user.email '<1132246821@pfur.ru>'
```

Рис. 3: Предварительная конфигурация git

Настраиваю utf-8 в выводе сообщений git для корректного отображения символов (рис. 4).

```
sultan@sultan-BOHB-WAX9:~$ git config --global core.quotepath false
```

Рис. 4: Настройка кодировки

Задаю имя «master» для начальной ветки (рис. 5).

```
sultan@sultan-BOHB-WAX9:~$ git config --global init.defaultBranch master
```

Рис. 5: Создание имени для начальной ветки

Задаю параметр autocrlf со значением input, так как я работаю в системе Linux, чтобы конвертировать CRLF в LF только при коммитах (рис. 6). CR и LF–это символы, которые можно использовать для обозначения разрыва строки в текстовых файлах.

```
sultan@sultan-BOHB-WAX9:~$ git config --global core.autocrlf input
```

Рис. 6: Параметр autocrlf

Задаю параметр safecrlf со значением warn, так Git будет проверять преобразование на обратимость (рис. 7). При значении warn Git только выведет предупреждение, но будет принимать необратимые конвертации.

```
sultan@sultan-BOHB-WAX9:~$ git config --global core.safecrlf warn
```

Рис. 7: Параметр safecrlf

4.3 Создание SSH-ключа

Для последующей идентификации пользователя на сервере репозиториев необходимо сгенерировать пару ключей (приватный и открытый). Для этого ввожу команду ssh-keygen -C “Имя Фамилия, work@email”, указывая имя владельца и электронную почту владельца (рис. 8). Ключ автоматически сохранится в каталоге ~/.ssh/.

```
sultan@sultan-BOHB-WAX9:~$ ssh-keygen -C "Парчиев Султан, 1132246821@pfur.ru"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/sultan/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/sultan/.ssh/id_ed25519
Your public key has been saved in /home/sultan/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:DURe3dkhI1KHhL3amlRjAZKwZY49FUyc6s4oW/Yffqc Парчиев Султан, 1132246821@pfur.ru
The key's randomart image is:
+--[ED25519 256]--+
|     ..=B*X=.= +.|
|      B+o*.++ = .|
|     o ++   o    |
|       ..o =     |
|      . S * .    |
|       . o .     |
|     o+ ..o      |
|   .o..o.o.. .   |
|   .o  ..o.Eo    |
+----[SHA256]-----+
```

Рис. 8: Генерация SSH-ключа

Xclip – утилита, позволяющая скопировать любой текст через терминал. Оказывается, в дистрибутиве Linux Kali ее сначала надо установить. Устанавливаю

xclip с помощью команды apt-get install с ключом -y от имени суперпользователя, введя в начале команды sudo (рис. 9).

```
sultan@sultan-BOHB-WAX9:~$ sudo apt-get install -y xclip
Чтение списков пакетов… Готово
Построение дерева зависимостей… Готово
Чтение информации о состоянии… Готово
Следующие НОВЫЕ пакеты будут установлены:
  xclip
Обновлено 0 пакетов, установлено 1 новых пакетов, для удаления отмечено 0 пакетов, и 31 пак
етов не обновлено.
Необходимо скачать 17,6 kB архивов.
После данной операции объём занятого дискового пространства возрастёт на 54,3 kB.
Пол:1 http://ru.archive.ubuntu.com/ubuntu noble/universe amd64 xclip amd64 0.13-3 [17,6 kB]
Получено 17,6 kB за 0с (605 kB/s)
Выбор ранее не выбранного пакета xclip.
(Чтение базы данных … на данный момент установлено 167907 файлов и каталогов.)
Подготовка к распаковке …/xclip_0.13-3_amd64.deb …
Распаковывается xclip (0.13-3) …
Настраивается пакет xclip (0.13-3) …
Обрабатываются триггеры для man-db (2.12.0-4build2) …
```

Рис. 9: Установка утилиты xclip

Копирую открытый ключ из директории, в которой он был сохранен, с помощью утилиты xclip (рис. 10).

```
sultan@sultan-BOHB-WAX9:~$ cat ~/.ssh/id_ed25519.pub | xclip -sel clip
```

Рис. 10: Копирование содержимого файла

Открываю браузер, захожу на сайт GitHub. Открываю свой профиль и выбираю страницу «SSH and GPG keys». Нажимаю кнопку «New SSH key» (рис. 11).

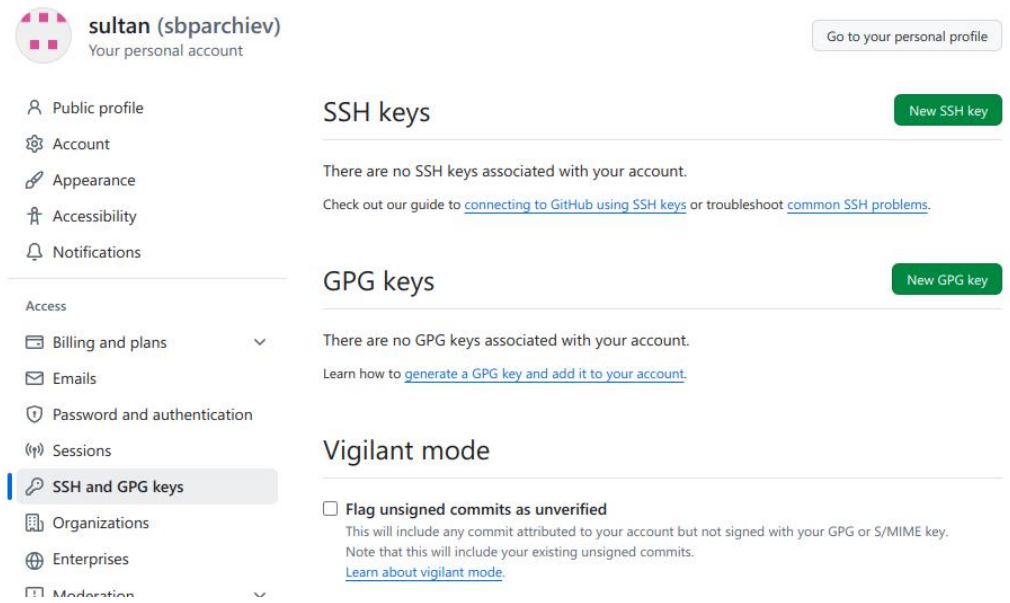

Рис. 11: Окно SSH and GPG keys

Вставляю скопированный ключ в поле «Key». В поле Title указываю имя для ключа. Нажимаю «Add SSH-key», чтобы завершить добавление ключа (рис. 12).

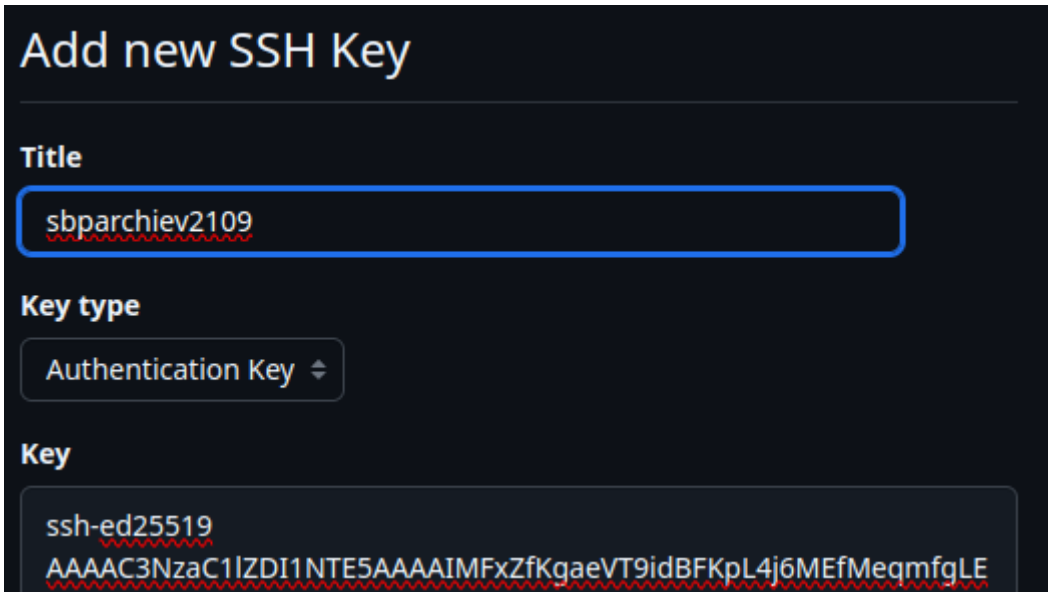

Рис. 4.12: Добавление ключа

4.4 Создание рабочего пространства и репозитория курса на основе шаблона

Закрываю браузер, открываю терминал. Создаю директорию, рабочее пространство, с помощью утилиты mkdir, благодаря ключу -p создаю все директории после домашней ~/work/study/2022-2023/“Архитектура компьютера” рекурсивно. Далее проверяю с помощью ls, действительно ли были созданы необходимые мне каталоги (рис. 13).

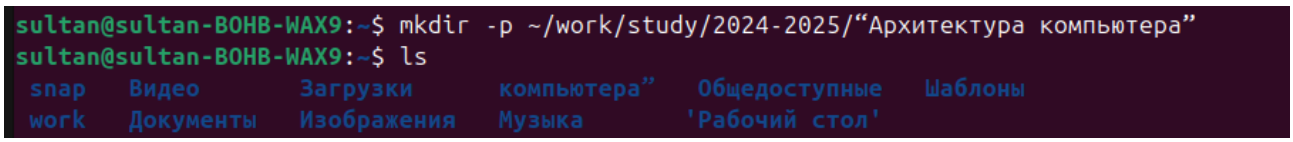

Рис. 13: Создание рабочего пространства

4.5 Создание репозитория курса на основе шаблона

В браузере перехожу на страницу репозитория с шаблоном курса по адресу https://github.com/yamadharma/course-directory-student-template. Далее выбираю «Use this template», чтобы использовать этотшаблон для своего репозитория (рис. 14).

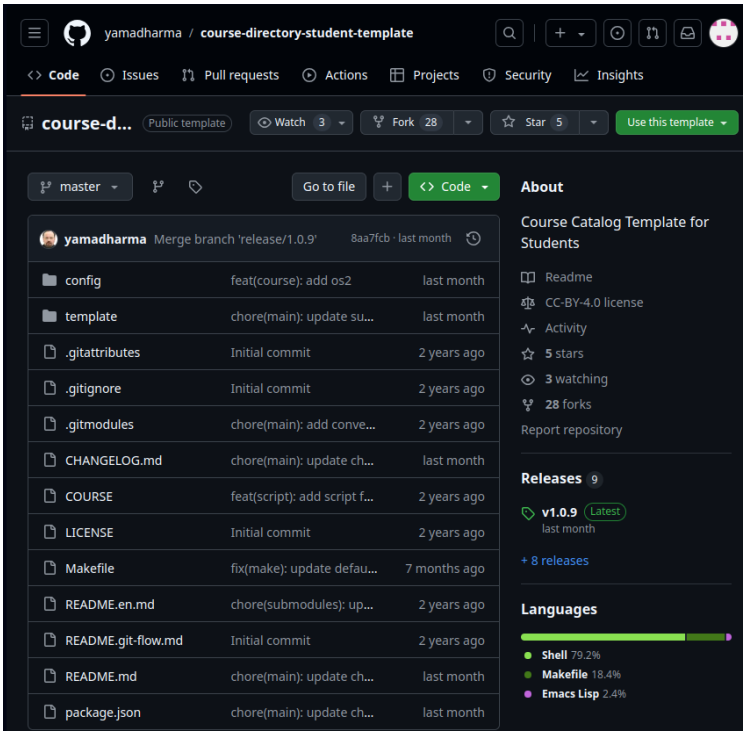

Рис. 14: Страница шаблона для репозитория

В открывшемся окне задаю имя репозитория (Repository name): study_2024–2025_arh-pc и создаю репозиторий, нажимаю на кнопку «Create repository from template» (рис. 15).

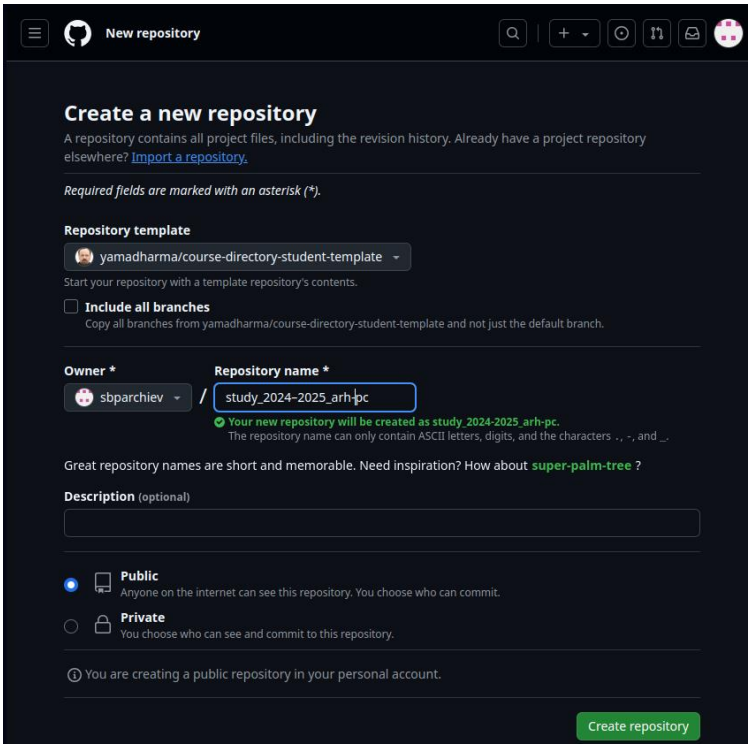

Рис. 15: Окно создания репозитория

Репозиторий создан (рис. 16).

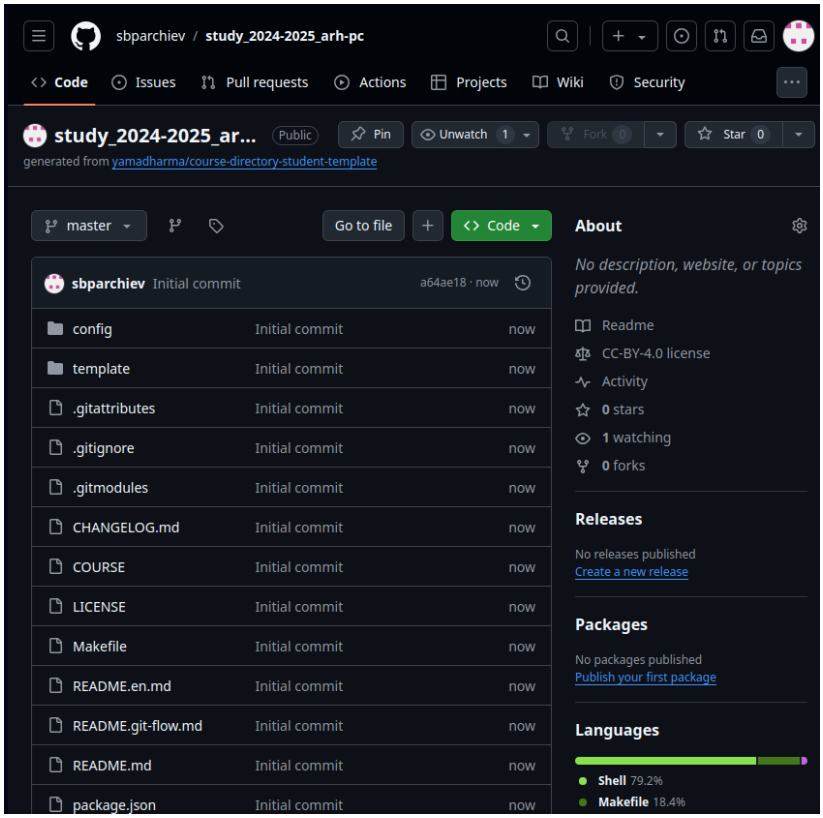

Рис. 16: Созданный репозиторий

Через терминал перехожу в созданный каталог курса с помощью утилиты cd (рис. 17).

```
sultan@sultan-BOHB-WAX9:~$ cd ~/work/study/2024-2025/'Архитектура компьютера'
sultan@sultan-BOHB-WAX9:~/work/study/2024-2025/Архитектура компьютера$
```

Рис. 17: Перемещение между директориями

Клонирую созданный репозиторий с помощью команды git clone –recursive git@github.com:/study_2024–2025_arh-pc.git arch-pc (рис. 18).

```
sultan@sultan-BOHB-WAX9:~/work/study/2024-2025/Архитектура компьютера$ git clone --recursiv
e git@github.com:sbparchiev/study_2024-2025_arh-pc.git arch-pc
Клонирование в «arch-pc»...
remote: Enumerating objects: 33, done.
remote: Counting objects: 100% (33/33), done.
```

Рис. 18: Клонирование репозитория

Копирую ссылку для клонирования на странице созданного репозитория, сначала перейдя в окно «code», далее выбрав в окне вкладку «SSH» (рис. 19).

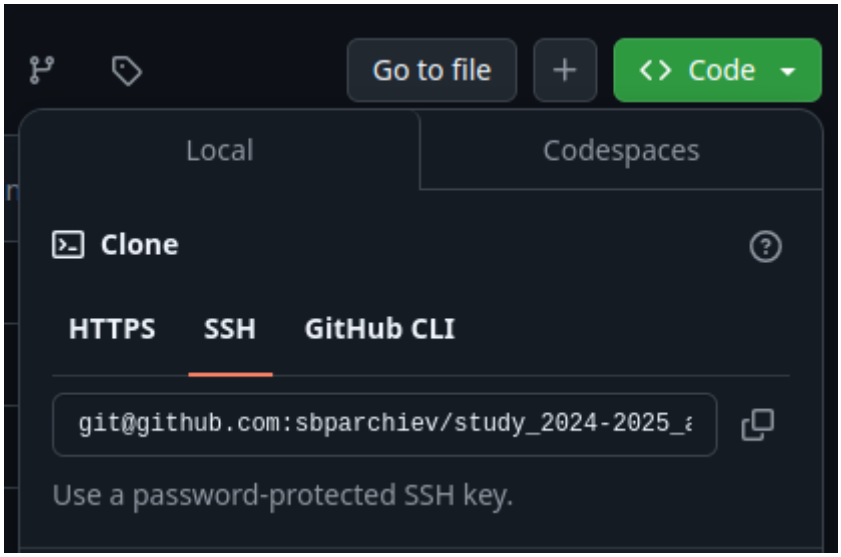

Рис. 19: Окно с ссылкой для копирования репозитория

4.6 Настройка каталога курса

Перехожу в каталог arch-pc с помощью утилиты cd (рис. 20).

```
sultan@sultan-BOHB-WAX9:~/work/study/2024-2025/Архитектура компьютера$ cd ~/work/study/2024
-2025/Архитектура\ компьютера/arch-pc
sultan@sultan-BOHB-WAX9:~/work/study/2024-2025/Архитектура компьютера/arch-pc$
```

Рис. 20: Перемещение между директориями

Удаляю лишние файлы с помощью утилиты rm (рис. 21).

```
sultan@sultan-BOHB-WAX9:~/work/study/2024-2025/Архитектура компьютера/arch-pc$ rm package.json
```

Рис. 21: Удаление файлов

Создаю необходимые каталоги (рис. 22).

```
sultan@sultan-BOHB-WAX9:~/work/study/2024-2025/Архитектура компьютера/arch-pc$ echo arch-pc >
COURSE
sultan@sultan-BOHB-WAX9:~/work/study/2024-2025/Архитектура компьютера/arch-pc$ make
Usage:
  make <target>

Targets:
  list                          List of courses
  prepare                       Generate directories structure
  submodule                     Update submules
```

Рис. 22: Создание каталогов

Отправляю созданные каталоги с локального репозитория на сервер: добавляю все созданные каталоги с помощью git add, комментирую и сохраняю изменения на сервере как добавление курса с помощью git commit (рис. 23).

```
sultan@sultan-BOHB-WAX9:~/work/study/2024-2025/Архитектура компьютера/arch-pc$ git add .
sultan@sultan-BOHB-WAX9:~/work/study/2024-2025/Архитектура компьютера/arch-pc$ git commit -am 'f
eat(main): make course structure'
[master 108cb5b] feat(main): make course structure
 2 files changed, 1 insertion(+), 14 deletions(-)
 delete mode 100644 package.json
sultan@sultan-BOHB-WAX9:~/work/study/2024-2025/Архитектура компьютера/arch-pc$ git push
Перечисление объектов: 5, готово.
Подсчет объектов: 100% (5/5), готово.
При сжатии изменений используется до 8 потоков
Сжатие объектов: 100% (2/2), готово.
Запись объектов: 100% (3/3), 278 байтов | 278.00 КиБ/с, готово.
Всего 3 (изменений 1), повторно использовано 0 (изменений 0), повторно использовано пакетов 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:sbparchiev/study_2024-2025_arh-pc.git
   a64ae18..108cb5b  master -> master
```

Рис. 23: Добавление и сохранение изменений на сервере. Выгрузка изменений на сервер

4.7 Выполнение заданий для самостоятельной работы

5 Выводы

Выполняя эту лабораторную работу, я создал свой репозиторий на GitHub, а также приобрел практические навыки по работе с системой git.

6 Список литературы

1. Архитектура ЭВМ